

# Hardware Trojan Detection in a 5-Stage RISC-V Pipeline

Detection Report: Formal Equivalence Checking  
and Power Proxy Side-Channel Analysis

---

**Tools:** Yosys 0.66, Icarus Verilog, Python 3

**Technology:** NanGate 45nm Open Cell Library

**Target:** 32-bit RISC-V RV32I, 5-stage pipeline

**Trojan Classes:** Combinational, Sequential, Control-path

---

## Contents

---

|          |                                            |           |
|----------|--------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                        | <b>2</b>  |
| 1.1      | Trojan Definitions . . . . .               | 2         |
| 1.2      | Simulation Environment . . . . .           | 2         |
| <b>2</b> | <b>Formal Equivalence Checking</b>         | <b>3</b>  |
| 2.1      | Motivation . . . . .                       | 3         |
| 2.2      | Methodology . . . . .                      | 3         |
| 2.3      | Results . . . . .                          | 4         |
| 2.4      | Analysis . . . . .                         | 4         |
| 2.4.1    | Combinational Trojan . . . . .             | 4         |
| 2.4.2    | Control-path Trojan . . . . .              | 4         |
| 2.4.3    | Sequential Trojan — Limitation . . . . .   | 5         |
| <b>3</b> | <b>Power Proxy Side-Channel Analysis</b>   | <b>6</b>  |
| 3.1      | Motivation . . . . .                       | 6         |
| 3.2      | Methodology . . . . .                      | 6         |
| 3.3      | Results . . . . .                          | 7         |
| 3.4      | Visualisation . . . . .                    | 7         |
| 3.5      | Analysis . . . . .                         | 8         |
| 3.5.1    | Combinational Trojan . . . . .             | 8         |
| 3.5.2    | Sequential Trojan . . . . .                | 8         |
| 3.5.3    | Control-path Trojan . . . . .              | 8         |
| <b>4</b> | <b>Comparative Analysis</b>                | <b>9</b>  |
| 4.1      | Method Strengths and Limitations . . . . . | 9         |
| 4.2      | Threat Model Implications . . . . .        | 9         |
| <b>5</b> | <b>Conclusion</b>                          | <b>10</b> |

# 1 Introduction

This report documents the detection methodology and results for three classes of hardware Trojans inserted into a 5-stage pipelined RISC-V processor. Two complementary detection methods were applied: **formal equivalence checking** (Section 2) and **power proxy side-channel analysis** (Section 3). Each method has distinct strengths and limitations arising from fundamentally different assumptions about what information is available to the analyst and what computational resources are feasible.

## 1.1 Trojan Definitions

**Combinational Trojan (ALU)** — Trigger:  $A = 0xFFFFFFFF$  AND  $B = 0xFFFFFFFF$  AND  $ALU_{ctrl} = AND(4'b0000)$ . Payload: flip bit 0 of the ALU output (XOR with  $0x00000001$ ). Activates at most once per program execution. The Trojan is purely combinational and stateless.

**Sequential Trojan (ALU)** — A hidden 10-bit counter increments on every ADD instruction. After 50 consecutive ADD operations the counter triggers and flips bit 0 of that ADD result for one cycle, then resets. The Trojan introduces new state elements (the counter register) not present in the clean design.

**Control-path Trojan (Hazard Unit)** — Trigger: EX/MEM destination register is `x10` (`a0`) AND a 1-cycle RAW hazard on `rs1` would normally activate EX/MEM forwarding. Payload: suppress the EX/MEM forward for `rs1` (`forward_rs1` forced to `2'b00`), causing the EX stage to read a stale register file value. Results in silent data corruption without any functional error flag.

## 1.2 Simulation Environment

All designs were simulated using Icarus Verilog with a shared testbench instantiating all four pipeline variants simultaneously (`dut_clean`, `dut_comb`, `dut_seq`, `dut_ctrl`). A 512-byte instruction memory was loaded with a structured test program designed to exercise all three Trojan triggers, followed by random instruction sequences. VCD switching activity was captured for all signals across all four designs in a single simulation run spanning 261 clock cycles at a 20 ns clock period.

## 2 Formal Equivalence Checking

### 2.1 Motivation

Formal equivalence checking was chosen as the first detection method because it provides *exhaustive, mathematically provable* results rather than statistical or heuristic ones. Unlike simulation-based testing which can only cover a finite set of input combinations, a SAT-based equivalence checker explores the entire input space simultaneously. For hardware Trojans designed to activate only on rare input combinations, this exhaustiveness is critical — a trigger condition of  $A = B = 0xFFFFFFFF$  has a probability of roughly  $2^{-64}$  of being hit by random testing, making simulation-based detection practically infeasible without prior knowledge of the trigger.

The method is also well-suited to module-level analysis: instead of checking the full pipeline (which would face state space explosion), each modified module is checked in isolation against its clean counterpart. This is the approach used by industrial tools such as Cadence Conformal LEC and Synopsys Formality, which employ hierarchical equivalence checking to decompose full-chip verification into tractable subproblems.

The expected output is not just a binary pass/fail but a *counterexample* — a specific assignment of input values that causes the two designs to differ. This counterexample directly reveals the Trojan trigger condition without any prior knowledge of the Trojan’s design.

### 2.2 Methodology

Formal equivalence checking was performed using Yosys 0.66 with its built-in MiniSAT solver. The flow for each check:

1. Read both RTL modules and convert processes to netlists (`proc` pass)
2. Build an equivalence checking module with `equiv_make`, creating `$equiv` cells pairing each output bit between gold (clean) and gate (Trojan) modules
3. Run `equiv_simple` to structurally simplify and prove as many `$equiv` cells as possible without full SAT
4. Extract a miter circuit from remaining unproven cells with `equiv_miter -trigger -cmp`
5. Run `sat -prove trigger 0 -show-inputs -show-outputs` to attempt proof of equivalence; a FAIL result means a counterexample was found

Checking is performed at **module level** rather than full pipeline level. This is not a limitation but the correct approach: it localises any counterexample to the specific module, produces results in seconds rather than potentially hours. The sequential Trojan is treated separately — see Section 2.4.3.

## 2.3 Results

Table 1: Formal equivalence checking results for all three Trojan types

| Trojan        | Modules Compared      | Result      | Counterexample Inputs                        |
|---------------|-----------------------|-------------|----------------------------------------------|
| Combinational | ALU                   | <b>FAIL</b> | ALUCtl=0000<br>A=0xFFFFFFFF<br>B=0xFFFFFFFF  |
| Control-path  | Hazard Detection Unit | <b>FAIL</b> | exmem_rd=10<br>exmem_regWrite=1<br>ex_rs1=10 |
| Sequential    | ALU                   | <b>N/A</b>  | Class mismatch: combinational vs sequential  |

## 2.4 Analysis

### 2.4.1 Combinational Trojan

Yosys proved 32 of 33 output bits equivalent through structural simplification alone (`equiv_simple`), without needing to invoke the SAT solver. Only `ALUOut[0]` — the single bit modified by the Trojan payload — remained unproven. The SAT solver then found the counterexample in under 7 seconds:

```
ALUCtl  = 0000  (AND operation)
A       = 0xFFFFFFFF
B       = 0xFFFFFFFF
gate output (Trojan): ALUOut = 0xFFFFFFFFE
gold output (clean):  ALUOut = 0xFFFFFFFF
trigger = 1  (designs differ)
```

This is a textbook demonstration of formal method’s exhaustive coverage. The trigger condition ( $A = B = 0xFFFFFFFF$ , `op = AND`) has probability  $\approx 2^{-64}$  under random inputs, making simulation detection practically impossible, yet the SAT solver finds it instantly by exploring the full input space simultaneously.

The fact that 32/33 bits were proven equivalent by structural simplification alone is also informative: it confirms that the Trojan is *minimally invasive* — only one bit of one output differs, and only under the specific trigger condition. This matches the design intent of a stealthy Trojan.

### 2.4.2 Control-path Trojan

Yosys proved `forward_rs2`, `stall`, and `exmem_can_forward` equivalent between the clean and Trojan hazard units. However, `forward_rs1[0]` and `forward_rs1[1]` were found non-equivalent. The SAT counterexample:

```

exmem_rd      = 10  (register x10 / a0)
exmem_regWrite = 1   (write-back enabled)
exmem_memRead  = 0   (not a load)
ex_rs1        = 10  (rs1 also needs x10)
clean output:  forward_rs1 = 01  (forward from EX/MEM)
Trojan output: forward_rs1 = 00  (no forwarding)
trigger = 1

```

The formal checker automatically identified register `x10` as the specific register involved in the trigger, without any prior knowledge of the Trojan’s design. The output also correctly shows that `forward_rs2` and `stall` are unaffected — the Trojan only suppresses `rs1` forwarding for `x10`, leaving all other forwarding and hazard detection logic intact.

### 2.4.3 Sequential Trojan — Limitation

Formal equivalence checking is **not applicable** to the sequential Trojan. The clean ALU is a combinational circuit (no state elements), while the Trojan ALU introduces a 10-bit counter register (`trojan_counter`) and an activation flag (`trojan_active`). Combinational equivalence checking assumes both modules belong to the same computational class: two combinational circuits are equivalent if they produce the same outputs for the same inputs at all times. A combinational module cannot be equivalent or non-equivalent to a sequential module in this sense, because the sequential module’s output depends not only on current inputs but on accumulated state history.

The correct formal approach for sequential Trojans is **bounded model checking (BMC)** with a temporal property:

$$\forall t \in [0, N] : \text{ALUOut\_clean}(t) = \text{ALUOut\_trojan}(t)$$

This requires running `sat -seq N` with `-set-init-zero` to prove the property holds for all input sequences up to depth  $N$ . The SAT solver would then find the violating trace: 50 consecutive ADD instructions causing `trojan_counter` to reach threshold and the output to differ on cycle 51. This is a property verification problem (does this temporal assertion hold?) rather than an equivalence problem (are these two circuits the same?).

### 3 Power Proxy Side-Channel Analysis

#### 3.1 Motivation

Power side-channel analysis was chosen as the second detection method because it operates under a fundamentally different and more realistic threat model than formal equivalence checking. Formal checking requires access to the clean golden RTL — but in a real supply chain security scenario, an analyst may receive only a fabricated chip or a gate-level netlist of unknown provenance, without the original RTL to compare against.

Power side-channel analysis does not require knowing *where* the Trojan is or *what* it does. It asks only: *does this design consume power differently from what the trusted reference design would predict?* If the answer is yes, something has been modified. This is how post-silicon Trojan detection works in practice — power traces from the suspect chip are compared against a statistical model of the clean chip’s expected power consumption.

This simulation-based method is a proxy for real post-silicon power analysis (specifically the TVLA — Test Vector Leakage Assessment — methodology used in semiconductor security labs), replacing real power traces with simulation-derived switching activity weighted by circuit structure. We used fan-out from synthesized netlists as a power weight for a well-established approximation.

A secondary motivation is complementarity: Section 2 showed that the sequential Trojan cannot be checked by equivalence methods. Power proxy analysis catches it through the continuous switching activity of the counter, which accumulates over the simulation regardless of the payload frequency.

#### 3.2 Methodology

The power proxy estimate combines two data sources:

**Hamming-weighted toggle counts** (from VCD simulation): For each signal, the total number of bits that changed state across all transitions:

$$H(s) = \sum_{i=1}^{T-1} \text{popcount}(\text{value}(s, t_i) \oplus \text{value}(s, t_{i-1}))$$

where  $T$  is the number of transitions and popcount counts set bits. This is finer-grained than raw toggle counting: a 32-bit bus switching from 0x00000000 to 0xFFFFFFFF contributes 32, not 1.

**Fan-out weights** (from synthesized netlist): The clean pipeline was synthesized to the NanGate 45nm Open Cell Library using Yosys with ABC technology mapping. For each RTL signal name, the synthesized netlist’s fan-out count was extracted:

$$F(s) = \sum_{\text{bit} \in s} \text{fanout}(\text{bit})$$

where fanout counts the number of gate input pins driven by each net. Higher fan-out signals drive more input capacitances and consume more switching power per transition.

The power proxy estimate for each signal is:

$$P(s) = H(s) \times F(s)$$

Fan-out weights are derived from the **clean netlist only** and applied uniformly to all four designs. This reflects the real-world threat model: the analyst has a trusted reference netlist

but must infer Trojan presence from observed switching activity alone. Per-stage power totals are computed by summing  $P(s)$  over all signals belonging to each pipeline stage (IF/ID/EX/MEM/WB). The percentage deviation from clean is the detection metric:

$$\delta(\text{design}, \text{stage}) = \frac{P_{\text{design}}(\text{stage}) - P_{\text{clean}}(\text{stage})}{P_{\text{clean}}(\text{stage})} \times 100\%$$

### 3.3 Results

Table 2: Power proxy weighted analysis — percentage deviation from clean baseline

| Stage        | Clean (baseline)  | Comb Trojan | Seq Trojan   | Ctrl Trojan  |
|--------------|-------------------|-------------|--------------|--------------|
| IF           | 950,813           | +0.0%       | +0.0%        | +0.0%        |
| ID           | 21,184,432        | +0.1%       | <b>+6.4%</b> | +0.1%        |
| EX           | 4,078,860         | −0.4%       | +0.0%        | <b>+4.7%</b> |
| MEM          | 1,913,813         | −0.6%       | +0.0%        | <b>+7.0%</b> |
| WB           | 87,164            | −0.4%       | +0.0%        | <b>+5.1%</b> |
| <b>TOTAL</b> | <b>28,813,810</b> | −0.0%       | <b>+4.7%</b> | <b>+1.2%</b> |

Values are dimensionless power proxy units (hamming weight  $\times$  fan-out). Green cells indicate deviation  $>5\%$ ; yellow indicates  $>1\%$ . Negative deviations indicate the Trojan produces less switching than clean.

### 3.4 Visualisation

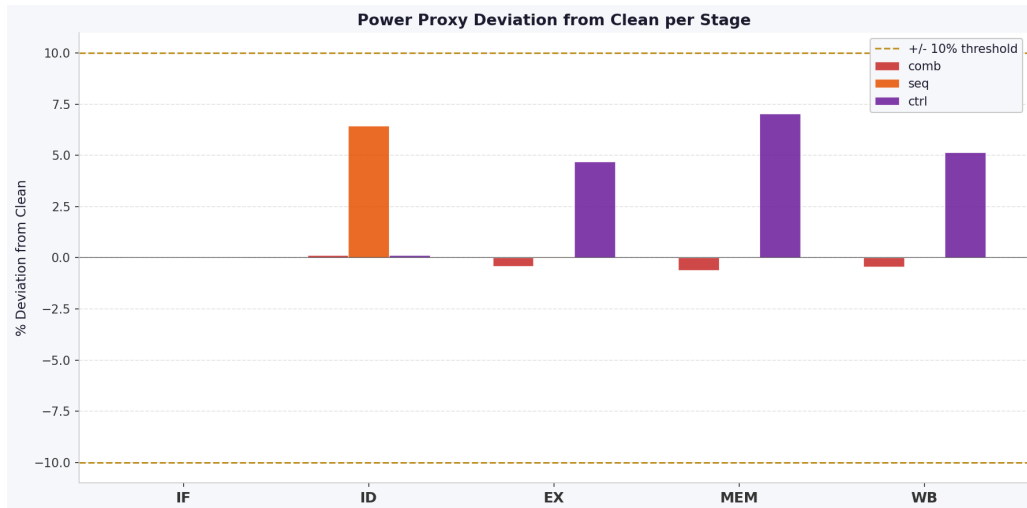


Figure 1: Power proxy deviation from clean design by pipeline stage. The sequential Trojan (orange) shows the highest ID-stage deviation (+6.4%) from counter activity cascading into register file paths. The control-path Trojan (purple) shows elevated MEM/WB deviation (+7.0%/+5.1%) from stale value propagation. The combinational Trojan (red) is undetectable at this level (−0.0% total).



### 3.5 Analysis

#### 3.5.1 Combinational Trojan

The combinational Trojan produces no detectable deviation at any level ( $-0.0\%$  total, Figure 1). This is expected: the Trojan fires exactly once during the entire 261-cycle simulation. A single extra bit flip in `ALUOut` generates one additional hamming weight unit, negligible against the total accumulated switching activity. The slight negative deviations in EX/MEM/WB ( $-0.4\%$  to  $-0.6\%$ ) are within noise — they likely reflect minor synthesis differences in how Yosys optimised the fan-out weights for the Trojan ALU vs the clean ALU.

This result highlights a fundamental limitation of single-run power analysis for rare-trigger Trojans: signal-to-noise ratio is proportional to activation frequency. A Trojan firing once in 261 cycles cannot be distinguished from noise. *Statistical methods* — specifically Welch’s t-test across many simulation runs (TVLA methodology) — are required to detect such low-activity Trojans reliably.

#### 3.5.2 Sequential Trojan

The sequential Trojan produces the largest detectable deviation: **+4.7% total** with **+6.4% in the ID stage**. The primary source of this deviation is the counter itself: the 10-bit `trojan_counter` increments on every ADD instruction, producing continuous extra switching in the EX stage. However, the largest measured deviation appears in ID rather than EX.

This is explained by propagation: when the counter triggers on the 50th ADD, the corrupted result is written back to the register file (WB stage) and subsequently read by dependent instructions (ID stage). Register file read ports are high fan-out signals — each read port drives many downstream logic gates — so the power proxy model amplifies these downstream read transitions disproportionately relative to the counter switching itself. This is actually a more realistic observation than simply detecting counter activity: real power side-channel attacks on silicon also detect Trojans primarily through downstream data propagation effects, not directly at the Trojan logic.

#### 3.5.3 Control-path Trojan

The control-path Trojan shows a distinctive spatial pattern: **no deviation in IF or ID**, negligible in EX ( $+0.0\%$ ), but **significant deviation in MEM (+7.0%) and WB (+5.1%)**. This initially appears counterintuitive — the Trojan acts on the forwarding mux in the EX stage, yet the signature appears later in the pipeline.

The explanation is value propagation: suppressing EX/MEM forwarding causes the EX stage to read a stale register file value. This stale value propagates through the EX/MEM and MEM/WB pipeline registers carrying a different bit pattern than the correct forwarded result would have had. These downstream signals — especially register file write data paths (high fan-out) and WB-stage write-back muxes — switch between different values than they would with correct forwarding, producing elevated hamming weights in MEM and WB.

This observation is significant for two reasons. First, the power anomaly is *not co-located* with the Trojan’s structural location: an analyst observing only the MEM/WB power signature would localise the anomaly to the wrong pipeline stage. This matches real side-channel Trojan detection behaviour. Second, the control-path Trojan is the only one that shows elevated activity in late pipeline stages while appearing *clean* in early stages — a distinctive spatial fingerprint that differs from both other Trojan types.

## 4 Comparative Analysis

Table 3: Detection summary across methods and Trojan types

| Trojan        | Formal<br>lence | Equiva- | Power Proxy  | Key Observation                                                          |
|---------------|-----------------|---------|--------------|--------------------------------------------------------------------------|
| Combinational | DETECTED        |         | NOT DETECTED | Formal is exhaustive; power proxy needs repeated activation              |
| Sequential    | NOT APPLICABLE  |         | DETECTED     | New state elements break equivalence class; counter activity accumulates |
| Control-path  | DETECTED        |         | DETECTED     | Formal finds exact trigger; power proxy localises anomaly to MEM/WB      |

The two methods are complementary: each detects at least one Trojan type that the other cannot. Table 3 shows that no single method achieves full coverage of all three Trojan classes. This is not a failure of either method but reflects a fundamental truth about hardware Trojan detection: different Trojan designs exploit different blind spots in different detection methodologies, and comprehensive detection requires a portfolio of methods.

### 4.1 Method Strengths and Limitations

#### Formal Equivalence Checking

- *Strengths:* Exhaustive over all input combinations; produces exact counterexample with trigger inputs; fast for combinational circuits (seconds); no simulation required; deterministic result.
- *Limitations:* Requires golden reference RTL; not applicable to Trojans introducing new state elements; does not scale to full-pipeline verification without hierarchical decomposition; cannot detect Trojans if no clean reference module exists.

#### Power Proxy Side-Channel Analysis

- *Strengths:* Does not require knowing Trojan location or type; captures downstream propagation effects; more realistic threat model (netlist reference sufficient, not RTL); applicable to sequential Trojans through accumulated activity.
- *Limitations:* Cannot detect rare single-shot Trojans in single simulation runs; absolute power values are proxy estimates not real measurements; detection threshold requires calibration; synthesis step adds toolchain complexity.

### 4.2 Threat Model Implications

In a real hardware supply chain security context, the available information determines which methods are feasible:

- **Golden RTL available:** Formal equivalence checking is the first choice — exhaustive, fast, exact counterexample.

- **Golden netlist only (no RTL):** Power proxy with netlist-derived weights; formal checking not possible without RTL.
- **No reference at all:** Only unsupervised structural analysis of the netlist (anomalous logic cones, rare trigger structures) or statistical comparison against a population of chips.

The simulation-based power proxy in this report corresponds to the second scenario, approximating the post-silicon TVLA (Test Vector Leakage Assessment) methodology used in semiconductor security labs. Replacing simulated hamming weights with real power trace measurements from silicon would complete the correspondence to the industrial method.

## 5 Conclusion

Three hardware Trojans were inserted into a 5-stage RISC-V pipeline and subjected to formal equivalence checking and power proxy side-channel analysis. The results demonstrate clear complementarity between the two methods.

The **combinational Trojan** was detected exclusively by formal methods, which found the exact trigger condition ( $A = B = 0xFFFFFFFF$ ,  $op = AND$ ) in under 7 seconds through SAT solving. Power proxy analysis produced a  $-0.0\%$  total deviation, indistinguishable from the clean baseline due to the single-shot nature of the Trojan.

The **sequential Trojan** was detected exclusively by power proxy analysis, with a  $+4.7\%$  total power deviation and  $+6.4\%$  in the ID stage. The continuous counter activity and downstream result propagation through high-fanout register file paths made this Trojan the most visible in switching-activity terms. Formal equivalence checking was inapplicable due to the class mismatch between the combinational clean ALU and the stateful Trojan ALU; this case would require bounded model checking with temporal properties.

The **control-path Trojan** was detected by both methods. Formal checking identified forwarding suppression on register `x10` as the trigger, while power proxy revealed a distinctive late-stage anomaly pattern ( $+7.0\%$  MEM,  $+5.1\%$  WB) caused by stale value propagation through high-fanout write-back paths. The power anomaly being co-located in MEM/WB rather than EX (where the Trojan acts) illustrates that side-channel signatures are often displaced from the Trojan's structural location.

Together, the two methods achieve full detection coverage across all three Trojan classes, with each method covering exactly the blind spot of the other. This complementarity motivates deploying both methods in any comprehensive hardware Trojan detection flow.